

Nested-Logit 分层多智能体强化学习在机器人仓库实时任务分配中的应用

Xuchen He

Vijay K. Madiseti

摘要

我们提出了一个基于嵌套 Logit 的分层多智能体强化学习 (NL-HMARL) 框架, 用于机器人仓库中的实时任务分配。管理层首先选择一个任务巢 (task nest), 然后在选定的巢内选择具体任务, 这样既能捕获巢内任务的相关性, 又保持端到端可训练。我们形式化了问题定义, 给出了策略和训练目标, 提供了算法描述, 并证明推理复杂度与任务数量呈线性关系。通过在 6 个不同配置 (3 种难度级别 $\times$ 2 种环境规模) 下的系统性实验, 我们证明 NL-HMARL 在高任务密度、低价值差异的复杂协调场景下优于标准 Softmax-HMARL, 并在大规模环境中展现出更显著的优势。

关键词: 仓库自动化; 多智能体强化学习; 嵌套 Logit; 分层强化学习; 离散事件仿真

1 引言

1.1 背景

全球电子商务的增长和不断缩短的配送承诺已将大型配送中心转变为高度动态的信息物理系统, 其中部署了数百台自主移动机器人 (AMR) 和人工拣货员。在这种环境中, 一个核心瓶颈是**实时任务分配**: 决定哪个智能体应该执行哪个任务 (拣货、移动、充电等), 以最大化吞吐量并避免拥堵。形式化来说, 这个问题耦合了: (i) 任务的随机到达过程, (ii) 空间受限的路径规划域, (iii) 资源冲突 (如狭窄通道和共享充电站)。

1.2 现有方法的局限性

传统的基于规则的调度器或混合整数规划方法对完整知识做出了强假设, 并且通常只在粗粒度时间尺度上重新优化。当任务持续到达且系统状态变化速度超过优化器求解速度时, 这些方法就会遇到困难。

更近期的**多智能体强化学习 (MARL)** 消除了手工设计的启发式规则, 但遭受维度诅咒: 扁平化 MARL 必须探索一个随智能体和任务数量指数增长的联合动作空间。

分层 MARL (HMARL) 通过引入管理层-工人层结构缓解了这个问题；然而，大多数管理层依赖于分类策略，隐式地假设了**无关选项独立性 (IIA)**。当一个高优先级任务出现在某个区域时，这个假设会导致管理层重新权衡**所有**备选项——无论相关与否——从而增加了其他地方拥堵和资源闲置的风险。

1.3 核心思想

为了放松 IIA 假设同时保持分层分解，我们考虑为高层管理层配备 **Nested Logit (NL)** 选择机制。NL 结构首先选择一个**巢 (nest)**——例如，区域 A 中的所有拣货任务或所有充电任务——然后在该巢内选择具体任务。这种两阶段视角在巢内保留了相关备选项的关联性，并将不同巢中的任务视为基本独立，这与真实仓库中观察到的空间和功能分组相匹配。

设计和学习这样的 NL-HMARL 架构提出了几个开放性问题：

1. 如何从原始仓库状态定义巢？
2. 如何将 NL 选择概率整合到强化学习更新中？
3. 如何保持整体决策循环足够快以实现实时控制？

1.4 论文范围

本文的其余部分专注于：

1. 形式化 NL-HMARL 框架（第 3 节）
2. 概述其学习算法（第 3.3 节）
3. 讨论它如何与低层运动控制器接口（第 3.2 节）
4. 实证评估和定量对比（第 5 节）

我们通过在 6 个不同配置下的系统性实验，验证了 NL-HMARL 相对于标准 Softmax-HMARL 的优势，并深入分析了环境复杂度和规模对性能的影响。

2 背景与相关研究

仓库订单拣货已成为多智能体决策的展示领域：数十台自主移动机器人 (AMR) 和人工拣货员必须在狭窄通道中协作，同时新订单不断到达。关键挑战是**实时任务分配**——决定**谁**应该执行**哪个**任务，以保持高吞吐量并避免拥堵。研究人员提出了从手工设计的路径规则到基于学习的调度器等各种解决方案。

2.1 手工设计的路径启发式

早期的仓库研究提出了几何启发式方法，如 **S-Shape**、**Return** 和 **Largest-Gap** 策略 [1]。这些方法规划确定性的拣货员路径——例如从通道的一端进入并从另一端离开——以最小的计算量保证完整覆盖。

优势: 无需训练，实现简单，对实践者直观。

劣势: 假设任务独立；当拣货稀疏或通道为死胡同时会增加额外行走距离。

2.2 精确和近似运筹优化方法

为了超越单拣货员规则，研究者转向精确优化。**旅行商问题 (Traveling Salesman Problem, TSP) 变体**将仓库拣货建模为组合优化问题：给定一组拣货点位置，求解访问所有点并返回起点的最短路径。由于 TSP 通过穷举或分支定界等方法搜索所有可能的路径组合，理论上能够找到全局最优解。然而，TSP 是 NP-hard 问题，其计算复杂度随拣货点数量呈指数级增长 [2]。近似动态规划方案——如分区优化 DP 或加速 SKU 分类——在解质量和运行时间之间取得平衡。然而，大多数运筹优化模型必须在新订单到达时重新求解，这限制了它们在大型配送中心的实时可用性。这一局限性为基于学习的方法提供了动机：通过离线训练，学习型方法能够在推理阶段实现常数时间复杂度的实时决策。

2.3 基于学习的单智能体方法

随着深度强化学习的出现，**DQN 风格**的智能体被训练在网格抽象上进行一步一步的规划 [3]。这类智能体响应快速，但当多个机器人同时行动时面临爆炸性的联合动作空间。细粒度策略也存在近视振荡的风险，因为它们缺乏全局任务分配视角。

在本综述中，我们将**扁平化**非分层 MARL 归入此类，因为它在没有管理层的情况下直接选择低层动作。虽然这类方法理论上可行，但在大规模仓储环境中面临严重的可扩展性挑战。如 Canese 等人 [4] 指出，联合动作空间的维度随智能体数量呈指数增长 ($|A|^N$ ，其中 N 为智能体数量)，使得这类方法难以扩展到超过少数几个智能体。Bahrpeyma 和 Reichelt [5] 在智能工厂的综述中进一步强调，联合动作学习方法 (JAL) 的探索策略设计极其困难，因为其联合动作空间远大于独立学习方法，这在拥有数十台机器人的大规模仓库环境中尤为严峻。

注: 由于计算资源和时间限制，本文未实现 DQN 或扁平化 MARL 作为 baseline。我们的实验重点在于验证 NL-HMARL 相对于标准 Softmax-HMARL 的优势 (核心对比)，以及与经典规则方法 (S-Shape、Return、Optimal) 的性能对比。上述 DQN 和扁平化 MARL 的讨论仅作为文献综述，说明本研究的背景和动机。

2.4 分层 MARL

分层 MARL (HMARL) 拆分决策栈：一个**管理层**分配任务，**工人层**策略执行动作 [6]。

尽管样本效率高，但大多数高层管理层依赖于分类 softmax 选择，这嵌入了**无关选项独立性 (IIA)** 假设。IIA 意味着任意两个选项之间的相对选择概率不受其他选项的影响。在实际应用中，当一个高度相似的新任务出现时，softmax 会不成比例地降低相关任务的概率，即使新任务的加入不应影响原任务相对于其他无关任务的优先级。

示例：考虑三个任务的选择场景——任务 A 和 B 位于区域 1（效用分别为 2.0 和 1.8），任务 C 位于区域 2（效用 1.5）。在标准 Softmax 下， $P(A) \approx 41\%$ ， $P(B) \approx 34\%$ ， $P(C) \approx 25\%$ 。若新增一个与 A、B 相似的任务 D（区域 1，效用 1.9），Softmax 会将概率均匀分散： $P(A)$ 降至约 30%，而无关的 $P(C)$ 也从 25% 降至约 18%。在 Nested-Logit 结构下，A、B、D 在同一巢内竞争，C 作为不同巢的任务，其相对优先级受影响较小。我们假设这种机制是 NL-HMARL 在复杂环境下优于 Softmax 的原因之一，实验结果与此假设一致（见第 5 节），但本文未直接测量决策概率分布来验证该机制。

2.5 建模相关任务

运筹学文献使用 **Nested Logit (NL)** 结构建模相关选择，将相似备选项分组为“巢”[7]。放松 IIA 假设的方法还包括 Mixed Logit (混合 Logit) 和 Probit 模型，但我们选择 Nested Logit 的原因是：(1) NL 保持闭式概率表达式，便于计算梯度和端到端训练；(2) NL 的层次结构与仓库任务的空间-功能分组天然契合；(3) 相比 Mixed Logit 的蒙特卡洛积分或 Probit 的高维积分，NL 计算效率更高，适合实时决策。然而，据我们所知，NL 与在线 MARL 管理层的集成——特别是使用可学习的巢相异性参数——尚未在仓库任务分配领域的文献中出现。

2.6 研究空白与展望

上述综述揭示了三个开放问题：

1. 当任务到达密集时的实时可扩展性
2. 明确建模共享空间或资源的任务之间的相关性
3. 超越黑盒 softmax 的透明、鲁棒的高层决策

本文的其余部分建议将 NL 层嵌入 HMARL 管理层，产生一个两阶段的**选择巢** → **选择任务策略**，该策略学习效用权重和巢相异性，同时保留端到端强化学习更新。

3 提出的方法

3.1 问题建模

我们考虑一个大型机器人仓库，其中有一组自主移动机器人（AMR）和在线到达的任务流（拣货、补货、充电、检查）。设 $t \in \{0, 1, \dots\}$ 索引决策时刻。仓库状态表示为 s_t ，它聚合了：

- (i) 机器人运动学和电池电量
- (ii) 当前任务池 $\mathcal{T}_t = \{\tau_t^1, \dots, \tau_t^A\}$ 及其空间位置和优先级
- (iii) 布局和资源状态（通道占用、充电站队列）

我们将 $A = |\mathcal{T}_t|$ 表示可用任务数量，并将任务分组为一组巢 $\mathcal{G}_t = \{\mathcal{N}_t^1, \dots, \mathcal{N}_t^G\}$ ，每个巢捕获一个相关子集（例如，某个区域的所有拣货任务，所有充电任务）。设 $G = |\mathcal{G}_t|$ 为巢的数量。

我们采用分层控制方案：高层管理层为每个机器人选择一个任务（或选择空闲），低层工人策略执行运动动作。设管理层在时刻 t 的决策为选择巢 $m_t \in \{1, \dots, G\}$ ，然后选择任务 $i_t \in \mathcal{N}_t^{m_t}$ 。每个机器人 k 然后遵循工人策略 $\pi_\omega(\cdot | o_t^k, i_t)$ ，该策略将局部观测 o_t^k 和分配的任务映射到低层动作。环境返回标量奖励 r_t ，平衡吞吐量、延迟惩罚、路径长度、能量使用、碰撞和死锁。

我们的目标是最大化期望折扣回报：

$$J(\theta, \phi, \lambda, \psi) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

其中 $\gamma \in (0, 1)$ ， (θ, ϕ, λ) 参数化管理层策略， ψ 参数化用于方差减少的评论家/基线。工人策略采用固定的启发式算法，不参与学习。

3.2 Nested-Logit 分层架构

为了放松无关选项独立性假设同时保持完全可微，我们为管理层配备 Nested-Logit (NL) 两阶段策略。

任务效用和巢相异性 对于每个候选任务 $i \in \mathcal{T}_t$ ，我们定义一个可学习的效用：

$$u_i = u_\theta(s_t, i) \in \mathbb{R}$$

对于每个巢 m ，定义一个可学习的相异性参数 $\eta_m \in [\eta_{\min}, 1]$ 。我们通过 sigmoid 参数化 $\eta_m = \sigma(\lambda_m)$ 得到 $(0, 1)$ 范围的值，再裁剪至 $[\eta_{\min}, 1]$ （其中 $\eta_{\min} \approx 0.1$ ）以确保数值稳定

性，并学习 $\lambda_m \in \mathbb{R}$ 。定义巢包容值：

$$I_m = \log \sum_{j \in \mathcal{N}_t^m} \exp\left(\frac{u_j}{\eta_m}\right)$$

两阶段管理层策略 我们引入巢级分数 $b_m = b_\phi(s_t, m)$ 。NL 管理层首先采样一个巢，然后在其中采样一个任务，概率为：

$$\pi_{\text{nest}}(m|s_t) = \frac{\exp(b_m + \eta_m I_m)}{\sum_n \exp(b_n + \eta_n I_n)}$$

$$\pi_{\text{task}}(i|s_t, m) = \frac{\exp(u_i/\eta_m)}{\sum_{j \in \mathcal{N}_t^m} \exp(u_j/\eta_m)}$$

在我们的设计中，每个任务只属于一个巢（由其空间位置和任务类型唯一确定）。因此，选择任务 i 的联合管理层概率为：

$$\pi_M(i|s_t) = \pi_{\text{nest}}(m_i|s_t) \cdot \pi_{\text{task}}(i|s_t, m_i)$$

其中 m_i 表示任务 i 所属的唯一巢。

这种结构通过 η_m 捕获巢内备选项之间的相关性，同时保持不同巢中的任务基本独立。

工人层 给定分配的任务 i_t ，工人策略 $\pi_\omega(\cdot|o_t^k, i_t)$ 产生运动级动作，直到终止（成功、失败或中止），此时控制返回管理层重新分配。为简化训练并加速收敛，工人策略使用预定义的启发式导航算法（基于 A* 寻路），而非端到端学习。

3.3 策略表示和学习

我们采用简化的 Advantage Actor-Critic (A2C) 方法训练管理层策略。记 $\hat{A}_t = R_t - V_\psi(s_t)$ 为优势估计，其中 $R_t = r_t + \gamma V_\psi(s_{t+1})$ 为 1 步 TD 回报。管理层的对数概率分解为：

$$\log \pi_M(i_t|s_t) = \log \pi_{\text{nest}}(m_t|s_t) + \log \pi_{\text{task}}(i_t|s_t, m_t)$$

管理层损失为：

$$\begin{aligned} \mathcal{L}_M(\theta, \phi, \lambda) = & -\mathbb{E}[\hat{A}_t \log \pi_M(i_t|s_t)] \\ & - \beta_H H[\pi_{\text{nest}}(\cdot|s_t)] \\ & - \beta_H \sum_m H[\pi_{\text{task}}(\cdot|s_t, m)] \end{aligned}$$

其中 β_H 为熵权重。评论家最小化 $\mathcal{L}_V(\psi) = \mathbb{E}[(R_t - V_\psi(s_t))^2]$ 。

管理层参数 $\{\theta, \phi, \lambda, \psi\}$ 通过随机梯度下降/上升更新。NL 组件完全可微；梯度通过 I_m 、 η_m 和对数概率流动。

3.4 性质和理论分析

端到端可微性和相关性建模 NL 分解保持整个管道可微，同时允许 $\eta_m \in [\eta_{\min}, 1]$ 捕获巢内相关性。当 $\eta_m \rightarrow 1$ 时，模型接近所有任务上的 softmax；较小的 η_m 增加巢内替代性并减少跨巢干扰。

实时复杂度 回顾第 3.1 节定义的 A （任务数）和 G （巢数）。一次决策需要：计算效用 u_i 为 $O(A)$ ；计算 I_m 为按巢分组任务的单次遍历， $O(A)$ ；形成巢 logits 为 $O(G)$ 。因此总体推理复杂度为 $O(A + G)$ 。由于巢数量 G 通常远小于任务数量 A ，实际复杂度近似为 $O(A)$ 。通过适度批处理和缓存同一区域任务共享的特征，实际延迟保持与毫秒级调度兼容。

鲁棒性和稳定性 通过在第二阶段仅重新权衡选定巢中的备选项，管理层对仓库其他地方不相关的高效用异常值变得不那么敏感，在突发到达下提高了稳定性。巢选择和巢内选择上的熵防止过早坍缩到单个区域，并在训练早期鼓励探索。

4 实验设置

4.1 仿真环境

我们采用离散事件仓库仿真器，捕获订单到达、机器人运动学、通道拥堵、充电约束和 workstation 服务时间。环境规模和配置根据实验需求调整：12×12 规模使用 20-35 个拣货员，24×24 规模使用 40-70 个拣货员，配备相应数量的拣货工作站（3-8 个）。订单根据非齐次泊松过程到达，具有小时变化（高峰/非高峰乘数）。每个订单被分解为拣货任务，物品位置从拟合典型周转率轮廓的热图中采样。任务巢通过空间区域和紧急程度在线形成（4 个功能区域 × 2 种紧急度 = 8 个巢）。

机器人遵循简单的差速驱动动力学，最大速度 1.2 m/s，依赖工人层进行局部碰撞避免。充电从 20% 到 80% 电量需要 15 分钟。管理层每 2 秒或任务提前终止时重新规划。每个 episode 持续 1 个模拟小时；我们报告 32 个种子的平均值。

表1展示了完整的实验配置，包括三种难度级别的核心参数差异以及不同规模下的环境设置。

4.2 评估指标

我们使用标准吞吐量和响应性指标评估策略：

- 订单吞吐量（订单/小时）和任务完成率（%）
- 平均任务等待时间和 95% 尾部延迟（秒）

表 1: 实验配置参数

类别	参数	Config1-Easy	Config2-Medium	Config3-Hard
订单参数	紧急订单占比 (urgent_order_prob)	0.40	0.30	0.25
	订单物品数 (max_items)	1	2	3
	突发概率 (bursty_prob)	0.35	0.50	0.65
	突发规模 (burst_size)	[12, 28]	[18, 40]	[25, 60]
	订单基准率 (base_rate)	1,200	1,500	1,800
环境约束	区域容量 (zone_capacity)	[3,3,4,3]	[2,2,3,2]	[2,2,2,2]
	超容惩罚 (overcapacity_penalty)	-2.0	-5.0	-8.0
	拥堵系数 (congestion_mult)	0.7	0.5	0.4
12×12	拣货员数量	20	30	35
	工作站数量	5	4	3
	叉车数量	4	4	4
24×24	拣货员数量	40	60	70
	工作站数量	8	8	6
	叉车数量	8	12	14

- 拥堵时间（在接近阈值内的秒数）
- Episode 回报（未折扣累积奖励）和安全违规（近碰撞次数/小时）

我们还报告每个区域的平衡性（队列长度的变异系数）以量化空间负载平滑。

4.3 基线方法

我们将 NL-HMARL 与以下方法对比：

- **规则启发式:** S-Shape 和 Return 路径规划，这两种方法使用预定义的路径模式进行任务分配
- **贪心最近距离方法 (Greedy-Nearest)：** 基于最小曼哈顿距离的贪心分配策略，为每个空闲拣货员选择距离最近的待分配任务。注：表格中简称为”Optimal” 以保持简洁，但该方法并非全局最优解
- **基于 Softmax 的分层 MARL:** 与 NL-HMARL 架构相同，但管理层使用标准分类 softmax 而非 Nested-Logit 结构

学习型基线 (Softmax-HMARL) 使用相同的训练步数 (10,000 步) 和类似的超参数设置。规则方法无需训练。

4.4 实现细节

管理层使用两层 MLP（隐藏大小 256/128）用于效用和巢评分器； $\eta_m = \sigma(\lambda_m)$ ，初始 $\lambda_m = 0$ 。工人使用预定义的启发式导航算法（基于 A* 寻路），无需学习。

我们使用 Adam 训练管理层（学习率 $= 1 \times 10^{-3}$ ），熵权重 $\beta_H = 0.01$ ，折扣 $\gamma = 0.99$ 。每 8 个管理决策步骤更新一次参数。训练使用 10,000 步（约对应 5-10 个完整 episode，具体取决于环境规模）。巢在每次管理层决策时根据任务的空间区域和紧急程度重新计算（4 个功能区域 $\times$ 2 种紧急度 = 8 个巢）。选择这种巢结构的理由是：(1) 空间区域划分反映了仓库的物理布局，同一区域的任务在路径规划上具有天然相似性；(2) 紧急度划分捕获了任务价值的主要差异维度；(3) 8 个巢在表达能力和计算效率之间取得平衡——太少的巢无法充分捕获任务相关性，太多的巢会增加学习难度。

试点实验在 MacBook Pro (M1 Pro) 上本地运行，启用 PyTorch MPS；完整规模训练在 Google Colab 的 A100 GPU 上运行。仿真器受 CPU 限制；实际时间主要随 CPU 吞吐量扩展。

5 实验结果与分析

我们在 6 个不同配置下进行了完整的实验评估，涵盖 3 种难度级别（Config1-Easy、Config2-Medium、Config3-Hard）和 2 种环境规模（ 12×12 、 24×24 ）。每个配置下我们对 5 种方法：NL-HMARL、Softmax-HMARL、Optimal（基于最小距离的贪心最优路径）、Return 路径规划和 S-Shape 路径规划。评估指标包括累积奖励值（Raw Value）、完成任务数（Tasks）以及平均每任务价值。

本章节首先呈现整体性能对比，然后深入分析环境复杂度和规模对 NL-HMARL 性能的影响，最后讨论方法的局限性和适用场景。

5.1 整体性能对比

表2展示了所有方法在不同配置下的完整性能数据。在 6 个配置中，NL-HMARL 在 5 个配置下优于 Softmax-HMARL，总体胜率达到 83.3%，验证了 Nested-Logit 结构在处理层次化任务分配中的有效性。值得注意的是，虽然 NL-HMARL 在所有配置下都落后于规则方法（Optimal、Return、S-Shape），但这并不影响我们的核心研究目标——验证 Nested-Logit 结构相对于标准 Softmax 在多智能体层次化决策中的优势。规则方法的领先主要归因于其针对特定场景的优化设计，而学习型方法追求更强的通用性和适应性。

表 2: 各配置下所有方法性能对比

配置	方法	12×12 累积奖励	12×12 任务数	24×24 累积奖励	24×24 任务数
Config1-Easy	Return	35,939	172	41,539	257
	S-Shape	34,346	183	36,350	212
	Optimal	18,809	228	44,949	441
	Softmax-HMARL	10,555	141	12,706	199
	NL-HMARL	9,351	162	13,790	206
Config2-Medium	Optimal	56,207	405	51,483	557
	S-Shape	42,864	270	59,991	328
	Return	33,060	255	58,321	350
	NL-HMARL	16,197	190	25,959	300
	Softmax-HMARL	14,901	234	19,339	288
Config3-Hard	Optimal	63,129	464	65,753	633
	S-Shape	51,886	335	70,072	412
	Return	45,039	312	65,246	404
	NL-HMARL	20,118	235	30,144	340
	Softmax-HMARL	18,932	267	19,758	235

5.2 环境复杂度影响分析

三种难度配置的关键设计意图是创造不同程度的 IIA 问题场景。`urgent_order_prob` 参数控制高价值紧急任务的比例：Config1-Easy 设为 0.40（40% 高价值任务，优先级区分明显），Config2-Medium 设为 0.30，Config3-Hard 设为 0.25（仅 25% 高价值任务，75% 普通任务效用相近）。

NL 优势的核心条件：当任务效用分布趋于均匀（即缺乏明显的高/低优先级区分）时，IIA 问题更加严重。在这种场景下，Softmax 会在效用相近的任务间不合理地分散选择概率——添加一个新的相似任务会同时降低**所有**其他任务（包括不相关任务）的选择概率。而 NL 结构通过将空间相邻或功能相关的任务分组到同一巢内，使得巢内任务相互竞争而不影响巢间的相对优先级，从而缓解 IIA 问题。

我们的实验揭示了一个重要发现：NL-HMARL 相对于 Softmax-HMARL 的性能优势与环境复杂度和规模呈正相关关系。表3展示了 NL-HMARL 相对于 Softmax-HMARL 的性能优势汇总。

Config1-Easy 环境分析 在最简单的配置下（单物品订单、低拥堵、高紧急任务占比），Softmax-HMARL 在 12×12 规模下胜出 11.4%。这是唯一一个 NL-HMARL 表现不如 Softmax 的案例。分析原因：单物品订单意味着无需复杂的多步骤路径规划；低拥堵环境下，拣货员之间协调需求较小；高紧急任务占比（`urgent_order_prob` = 0.40）使得简单的贪心策略（优先选择高价值任务）已经非常有效；在这种场景下，IIA 问题并不突出，因为任务之间的替代性较弱。

表 3: NL-HMARL 相对于 Softmax-HMARL 的性能优势

难度级别	12×12 优势	24×24 优势	规模放大增益	胜率
Config1-Easy	-11.4%	+8.5%	+19.9pp	1/2
Config2-Medium	+8.7%	+34.2%	+25.5pp	2/2
Config3-Hard	+6.3%	+52.6%	+46.3pp	2/2
平均/总计	+1.2%	+31.8%	+30.6pp	5/6

然而值得注意的是，在 24×24 规模下，NL-HMARL 成功反超 8.5%，表明即使在简单环境中，规模放大也能激发 NL 结构的优势。

Config2-Medium 环境分析 中等复杂度环境下，NL-HMARL 开始显现明显优势，在 12×12 和 24×24 规模下分别领先 8.7% 和 34.2%。关键特征：多物品订单（max_items = 2）增加了路径规划复杂度；中等拥堵程度需要一定的拣货员协调；紧急任务占比降低（urgent_order_prob = 0.30），简单贪心策略开始失效；Nested 结构允许模型首先在巢层面进行粗粒度决策（选择哪个区域），然后在任务层面细化，这种层次化决策更适合中等复杂度的协调问题。

Config3-Hard 环境分析 最复杂环境下，NL-HMARL 的优势充分发挥，在 12×12 和 24×24 规模下分别领先 6.3% 和 52.6%。最佳表现出现在 Config3-Hard 24×24 配置：NL-HMARL 获得 30,144 累积奖励，而 Softmax 仅获得 19,758，领先 52.6%；任务完成数差距：NL-HMARL 完成 340 个任务，Softmax 仅完成 235 个，多完成 44.7%。

环境特征：多物品订单（max_items = 3）；高拥堵（tight zone_capacity 配置）；低紧急任务占比（urgent_order_prob = 0.25），无法简单“抓大放小”；高突发概率（bursty_prob = 0.65），订单到达更不规律。

在这种高度复杂的环境中，IIA 问题成为关键瓶颈。当多个任务都具有相似的价值时，Softmax 的 IIA 假设导致次优决策：即使添加或移除一个不相关的低价值任务，也可能显著改变高价值任务之间的选择概率。而 NL 结构通过巢层次，首先识别出相似任务的组（例如同一区域的任务），然后在组内进行选择，有效缓解了这一问题。

5.3 规模放大效应

如表3所示，从 12×12 规模到 24×24 规模，NL-HMARL 的平均优势从 +1.2% 大幅提升到 +31.8%，增益达 30.6 个百分点。更重要的是，在 24×24 规模下，NL-HMARL 实现了 100% 的胜率（3/3），而在 12×12 规模下仅为 66.7%（2/3）。

为什么规模放大会增强 NL 优势? 基于实验观察和理论分析, 我们提出以下可能的解释 (注: 这些是理论推测, 尚未通过专门的消融实验逐一验证):

1. **协调复杂度指数增长:** 从 12×12 (20-35 个拣货员) 到 24×24 (40-70 个拣货员), 拣货员数量增加了约 2-3 倍, 但潜在的任务分配组合数呈指数增长。在高密度场景下, 同一时刻可能有数十个拣货员竞争同一区域的任务, IIA 问题变得更加突出。
2. **层次化决策的表达优势:** 更大的环境意味着更多的任务和更复杂的状态空间。NL 的两层决策结构 (巢选择 $\rightarrow$ 任务选择) 相比 Softmax 的单层决策, 能够更有效地表达复杂的决策策略。
3. **区域划分的价值:** 24×24 环境下, 4 个功能区域 (存储区、打包区、充电区、通道) 的空间分布更加明显。NL 将任务按区域自然分组为巢, 这种结构化表达在大规模环境下更有价值。
4. **训练收敛效率:** 虽然两种方法使用相同的训练步数 (10,000 步), 但在大规模环境下, NL 的层次化结构可能带来更好的样本效率。

理论复杂度与推理效率对比 回顾第 3.4 节中的理论分析, 各方法的时间复杂度如下: NL-HMARL 和 Softmax-HMARL 的推理复杂度为 $O(A + G)$ (A 为任务数, G 为巢数), 由于神经网络结构固定, 实际推理时间近似 $O(1)$; S-Shape/Return 方法需要对每个拣货员遍历任务进行分配和路径排序, 复杂度为 $O(P \cdot T)$ (P 为拣货员数, T 为任务数); Optimal 方法基于全局距离矩阵搜索, 复杂度为 $O(P \cdot T \cdot W \cdot H)$ ($W \times H$ 为网格大小)。

表4展示了各方法在不同规模下的实际推理时间基准测试结果。

表 4: 各方法推理效率对比

方法	时间复杂度	12×12 (ms)	24×24 (ms)	规模增长率
NL-HMARL	$O(A + G) \approx O(1)$	0.057	0.057	$\approx 1 \times$
Softmax-HMARL	$O(A) \approx O(1)$	0.060	0.060	$\approx 1 \times$
S-Shape/Return	$O(P \cdot T)$	0.058	0.19	$\approx 3.3 \times$
Optimal	$O(P \cdot T \cdot W \cdot H)$	1.98	7.86	$\approx 4 \times$

核心发现: 学习方法 (NL-HMARL、Softmax) 的推理时间几乎不随环境规模变化, 从 12×12 到 24×24 均保持 $\sim 0.06\text{ms}/\text{决策}$ 。相比之下:

- S-Shape/Return 方法在 24×24 规模下推理时间增长 **3.3 倍** (从 0.058ms 增至 0.19ms)
- Optimal 方法在 24×24 规模下已比学习方法慢 **138 倍** (7.86ms vs 0.057ms)

这一结果揭示了学习方法在大规模实时系统中的关键优势：**一次性训练成本换取恒定推理时间**。对于需要毫秒级响应的仓库调度系统，这种特性具有重要的工程意义。

大规模电商仓库场景展望 实际大型电商仓库的规模远超本实验设置。以亚马逊、京东等头部电商的自动化仓库为例，典型配置为 100×100 网格甚至更大（约 10,000 平方米作业区域），配备 500-1000 名拣货员（人工或 AGV），每小时处理数万个订单。基于本实验的时间复杂度分析，我们对各方法在 100×100 规模下的推理性能进行外推估算：

- **NL-HMARL/Softmax-HMARL**: 由于神经网络结构固定,推理时间保持 $\sim 0.06\text{ms}/\text{决策}$, **与规模无关**。
- **S-Shape/Return**: 时间复杂度 $O(P \cdot T)$ 。假设 100×100 环境有 500 个拣货员和 1000 个待分配任务（约为 24×24 的 8-10 倍），推理时间预计增长至 **1-2ms/决策**, 比学习方法慢 **20-30 倍**。
- **Optimal**: 时间复杂度 $O(P \cdot T \cdot W \cdot H)$ 。 100×100 规模下，网格面积是 24×24 的 17 倍，叠加拣货员和任务数增长，推理时间预计达到**秒级**, **无法满足实时决策需求**。

这一展望表明：在真实大规模电商仓库场景下，学习方法的计算效率优势将从实验中观察到的数倍差距扩大到**数十倍甚至更多**。NL-HMARL 的恒定推理时间特性使其特别适合需要毫秒级响应的大规模实时调度系统。

6 结论

本文提出了 Nested-Logit 分层多智能体强化学习 (NL-HMARL) 框架，用于机器人仓库中的实时任务分配。通过 3 种难度级别和 2 种环境规模的系统性实验，我们得出以下核心结论：

(1) 学习方法具有计算效率优势: NL-HMARL 推理时间恒定 ($\sim 0.06\text{ms}/\text{决策}$)，不随环境规模增长；而 Optimal 方法在 24×24 规模下已比学习方法慢 138 倍。在 100×100 级别的大型电商仓库中，这一优势将扩大到数十倍以上。

(2) NL 结构在复杂场景下优于 Softmax: 当任务效用分布均匀 (IIA 问题严重) 时，NL 优势显现。在 Config3-Hard 24×24 配置下，NL-HMARL 比 Softmax 累积奖励高 52.6%，任务完成数多 44.7%。环境规模放大时，NL 胜率从 66.7% 提升至 100%。

局限性: 在简单环境中 NL 未展现优势；训练步数 (10,000 步) 较少；与精调的规则方法相比仍有性能差距。

未来工作: 增加训练步数、优化网络架构（注意力机制、图神经网络）、探索迁移学习、

验证更大规模 (100×100) 环境、以及巢结构的自动学习。

参考文献

- [1] H. D. Ratliff and A. S. Rosenthal, “Order-picking in a rectangular warehouse: A solvable case of the traveling salesman problem,” *Operations Research*, vol. 31, no. 3, pp. 507–521, 1983.
- [2] R. De Koster and E. S. Van der Poort, “Routing order pickers in a warehouse: A comparison between optimal and heuristic solutions,” *IIE Transactions*, vol. 30, pp. 469–480, 1998.
- [3] M. S. Alam, M. U. Khan, and A. Gunes, “Reinforcement learning-based multi-robot path planning and congestion management in warehouse order picking,” in *Proc. 26th International Multi-Topic Conference (INMIC)*, 2024, pp. 1–6.
- [4] L. Canese, G. C. Cardarilli, L. Di Nunzio, R. Fazzolari, D. Giardino, M. Re, and S. Spanò, “Multi-agent reinforcement learning: A review of challenges and applications,” *Applied Sciences*, vol. 11, no. 11, p. 4948, 2021.
- [5] F. Bahrpeyma and D. Reichelt, “A review of the applications of multi-agent reinforcement learning in smart factories,” *Frontiers in Robotics and AI*, vol. 9, p. 1027340, 2022.
- [6] A. Krnjaic, R. D. Steleac, J. D. Thomas, G. Papoudakis, L. Schäfer, A. W. K. To, K.-H. Lao, M. Cubuktepe, M. Haley, P. Børsting, and S. V. Albrecht, “Scalable multi-agent reinforcement learning for warehouse logistics with robotic and human co-workers,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 677–684.
- [7] K. E. Train, *Discrete Choice Methods with Simulation*, 2nd ed. Cambridge University Press, 2009.